Stack Application: Expression Conversion and Evaluation

Data Structures and its Applications

N S Kumar
Professor
Department of Computer Science &
Engineering (AI & ML)

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Stack Application: expression conversion and evaluation

a = b = c

The equivalent expression in postfix is

a) a b = c =

b) a b c = =

c) c b = a =

d) c b a = =

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 1: Stack Application: expression conversion and evaluation

a = b = c

The equivalent expression in postfix is

a) a b = c =

b) a b c = =

c) c b = a =

d) c b a = =

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Stack Application: expression conversion and evaluation

While evaluating an expression in postfix, we require

a) a stack of operators

b) a stack of operands

c) Stacks of operator and operands

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 2: Stack Application: expression conversion and evaluation

While evaluating an expression in postfix, we require

a) a stack of operators

b) a stack of operands

c) Stacks of operator and operands

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Stack Application: expression conversion and evaluation

In a postfix expression,

a) Left parenthesis has the highest precedence

b) Right parenthesis has the highest precedence

c) There are no parentheses

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 3: Stack Application: expression conversion and evaluation

In a postfix expression,

a) Left parenthesis has the highest precedence

b) Right parenthesis has the highest precedence

c) There are no parentheses

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Stack Application: expression conversion and evaluation

While converting an expression from infix to postfix,

a) Left parenthesis has the highest precedence in the input string

b) Left parenthesis has the highest precedence in the stack

c) Precedence of parentheses is same in the input string as well as in the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 4: Stack Application: expression conversion and evaluation

While converting an expression from infix to postfix,

a) Left parenthesis has the highest precedence in the input string

b) Left parenthesis has the highest precedence in the stack

c) Precedence of parentheses is same in the input string as well as in the stack

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Stack Application: expression conversion and evaluation

For this postfix expression $789 + *$, the corresponding infix expression is

a) $7 * 8 + 9$

b) $7 + 8 * 9$

c) $(7 + 8) * 9$

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Quiz 5: Stack Application: expression conversion and evaluation

For this postfix expression $789 + *$, the corresponding infix expression is

a) $7 * 8 + 9$

b) $7 + 8 * 9$

c) $(7 + 8) * 9$

d) None of these

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

stack

- Arithmetic expressions are easy for humans to write
- parsing them in a program can be complicated.
- Infix notation uses operators between operands: 3 + 4.
- Postfix notation puts operators after operands: 3 4 +.
- A stack makes postfix evaluation straightforward.

© Confidential Information of PES University. All Rights Reserved - 2026

Why Expression Evaluation Is Complicated

- BODMAS gives a familiar order: Brackets, Orders, Division, Multiplication, Addition, Subtraction.
- Programming languages add many precedence levels and associativity rules.
- Parentheses introduce nesting.
- A useful alternative is a notation in which operators can be processed as soon as they are encountered.

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Infix and postfix expressions

- 3 + 4 => 3 4 +
- 5 * 6 + 7 => 5 6 * 7 +
- 5 + 6 * 7 => 5 6 7 * +
- (3 + 4) * (5 + 6) => 3 4 + 5 6 + *
- Observe : no parentheses in postfix expression

© Confidential Information of PES University. All Rights Reserved - 2026

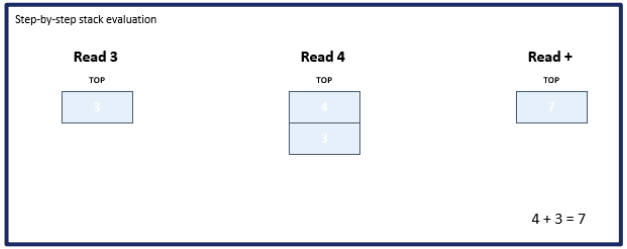
Postfix Evaluation Using a Stack

1. Create a stack of operands.
2. Read the postfix expression from left to right.
3. Operand → push it onto the stack.
4. Operator → pop the required operands, perform the operation, and push the result.
5. After the complete expression is processed, the stack contains exactly one value: the result.

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Example: 3 4 +

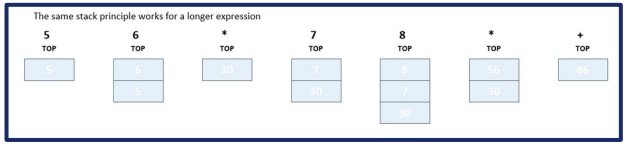


Final result = 7

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Example: 5 6 7 8 +



30 + 56 = 86 → Final result = 86

© Confidential Information of PES University. All Rights Reserved - 2026

Postfix Evaluation: Algorithm

Read the postfix expression.

Create and initialize a stack of integers.

```
For each token:
if token is an operand:
push(token)
else if token is an operator:
val2 = pop()
val1 = pop()
result = val1 operator val2
push(result)
```

```
At the end:
result = pop()
the stack should now be empty.
```


© Confidential Information of PES University. All Rights Reserved - 2026

Postfix Evaluation: order of evaluation

For + and *, operand order does not affect the result.

For – and /, it matters.

If the postfix expression is: 8 3 –

First pop gives val2 = 3; second pop gives val1 = 8.

Compute val1 – val2 = 8 – 3 = 5.

Never compute 3 – 8

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Conversion from infix to postfix

- Example: 5 * 6 + 7 * 8
- Postfix form: 5 6 * 7 8 * +
- Operands are sent directly to the output.
- Relative positions of the operands is never changed
- A stack of operators is required
- Operators are temporarily stored on a stack.
- The stack allows operators to be delayed

until their precedence requires them to be output

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Conversion from infix to postfix

- Associate a precedence value with each operator
- Higher the precedence, higher the value
- When an operator is encountered, we compare the precedence value of the operator in the string with that of the operator in the top of the stack
- While the first one is higher, push the operator into the stack
- Otherwise pop the operator from the stack and push the operator

in the string on to the stack

- When the input string is completely processed, pop all the operators from the stack

© Confidential Information of PES University. All Rights Reserved - 2026

Conversion from infix to postfix: Parentheses During Infix → Postfix

A left parenthesis is always pushed onto the operator stack.

It acts as a barrier: operators inside the parentheses are handled before operators outside.

When a right parenthesis is encountered, pop operators until the matching left parenthesis is found.

The parentheses themselves are not included in the postfix output.

Example: $3 * (4 + 5) * 6 \rightarrow 345 + * 6 *$.

© Confidential Information of PES University. All Rights Reserved - 2026

Stack Application: Expression Conversion and Evaluation

Summary

- Postfix notation removes the need for precedence rules during evaluation.
- A stack of operands is sufficient for postfix evaluation.
- Operators pop operands, compute, and push the result.
- Infix-to-postfix conversion uses a stack of operators.
- Precedence, associativity, and parentheses determine when operators are popped.
- The stack can be used through its interface without knowing whether it is implemented with an array or a list.

© Confidential Information of PES University. All Rights Reserved - 2026

THANK YOU

N S Kumar
Professor

Department of Computer Science & Engineering (AI & ML)